

Challenge -05

Objective

The objective of this task was to understand and implement AWS Lambda, API Gateway, and AWS Key Management Service (KMS). These services are commonly used in serverless architectures to build secure, scalable, and cost-effective applications without managing servers.

1. AWS Lambda

Concept

[AWS Lambda Documentation](#)

AWS Lambda is a **serverless compute service** that runs your code without managing servers.

Use Cases

- Backend APIs
- File processing
- Automation
- Scheduled tasks
- Event-driven applications

Hands-On Implementation

Step 1: Create Lambda Function

1. AWS Console → Lambda

2. Click **Create Function**

aws mohdabdulw Push ALB Log Create functi (1) WhatsApp Mohd Abdul Auto Scaling SCRUM-267 Domain Port Bright Future (55)

https://us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/create/function?firstrun=true&intent=authorFromScratch

aws Search [Alt+S]

Lambda > Functions > Create function

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

myFunctionName

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Node.js 24.x

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution** role under **Additional settings**.

3. Author from scratch

4. Function name:

HelloWorldFunction

5. Runtime:

Python 3.13

aws Search [Alt+S]

Lambda > Functions > Create function

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

HelloWorldFunction

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.14

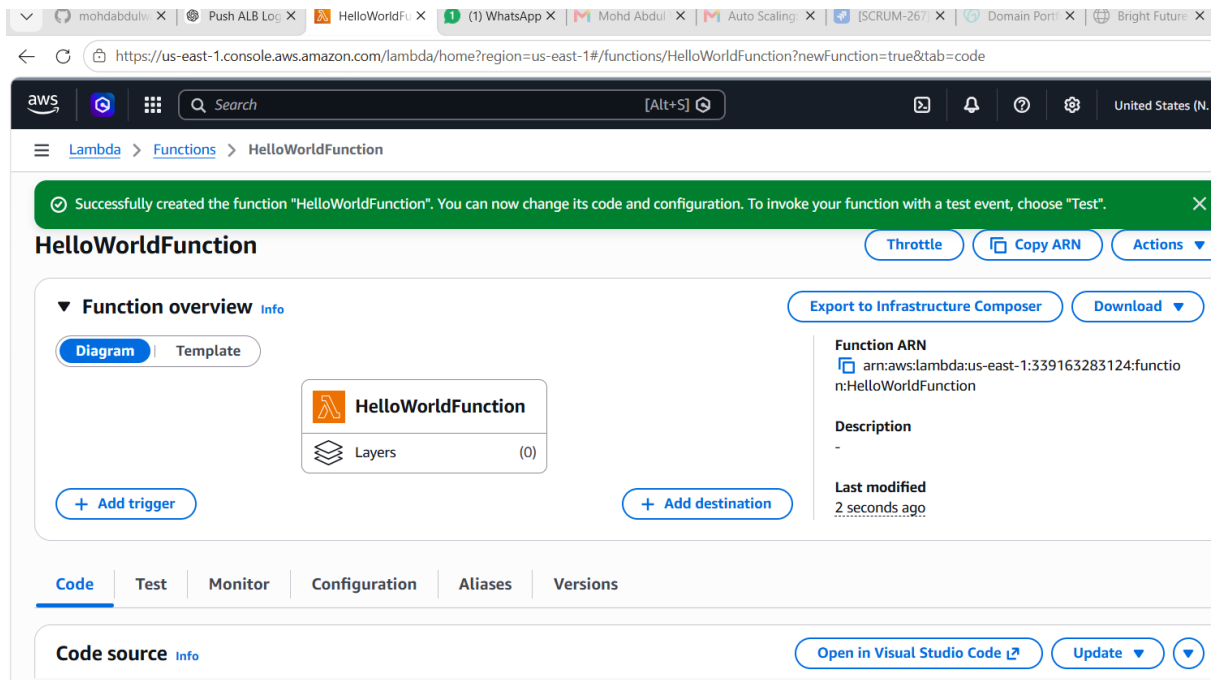
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution** role under **Additional settings**.

Custom settings [Info](#)

Durable execution - *new*

EC2 capacity provider - *new*

6. Create Function

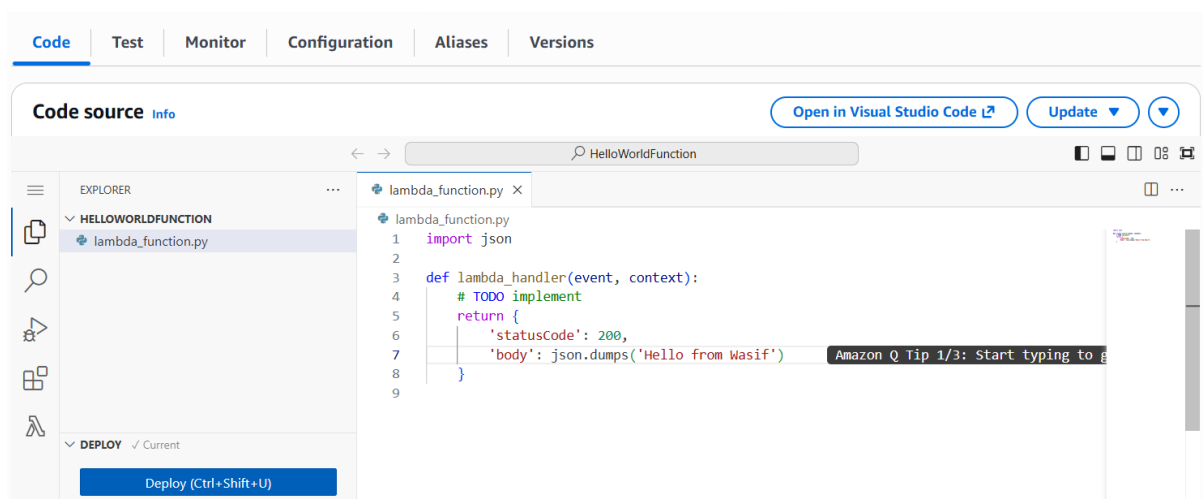


Step 2: Add Code

Replace existing code:

```
def lambda_handler(event, context):  
    return {  
        'statusCode': 200,  
        'body': 'Hello from Khadeer'  
    }
```

Click **Deploy**

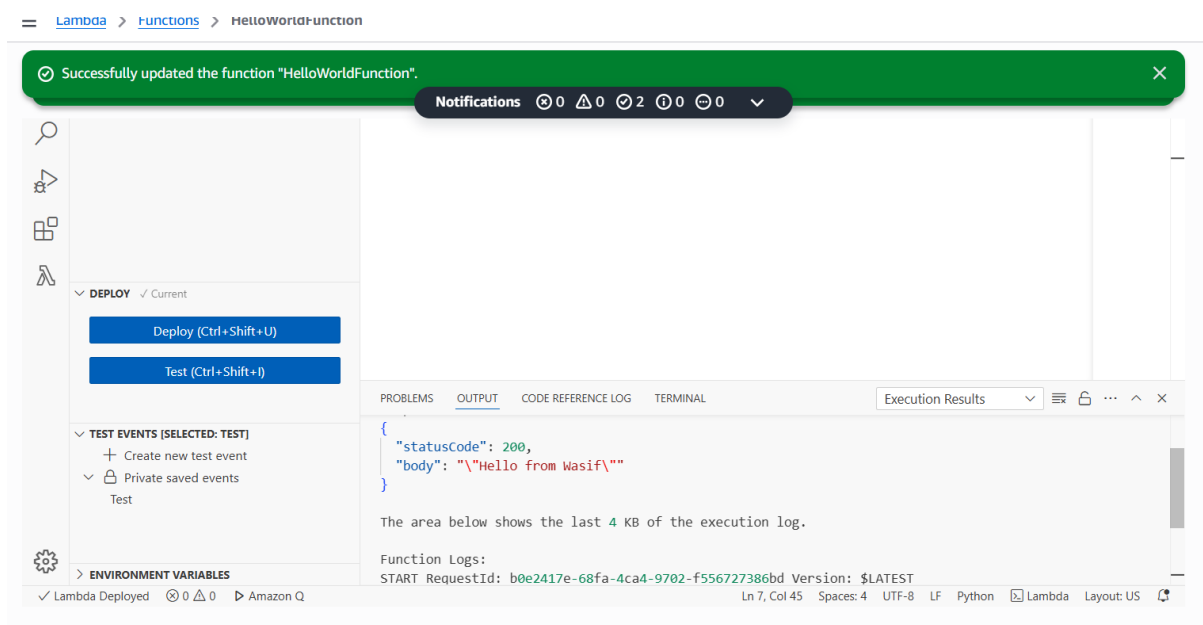


Step 3: Test Lambda

1. Click Test
2. Create Test Event
3. Save
4. Run Test

Expected Output:

```
{
  "statusCode": 200,
  "body": "Hello from Khadeer"
}
```



2. AWS API Gateway

Concept

[Amazon API Gateway Documentation](#)

API Gateway allows applications to expose REST APIs and connect them to backend services like Lambda.

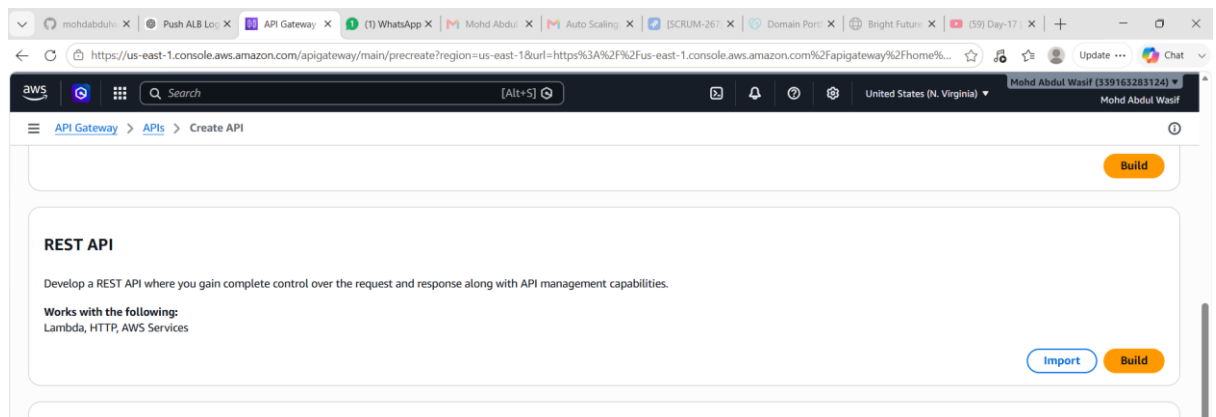
Use Cases

- REST APIs
- Mobile Backends
- Serverless Applications

Hands-On Implementation

Step 1: Create API

1. API Gateway
2. Create API
3. REST API
4. Build



Step 2: Configure API

API Name:

LambdaAPI

Create API

Step 3: Create Resource

Resource Name:

Hello

The screenshot shows the AWS API Gateway console during the 'Create HTTP API' process. The left sidebar indicates the current step is 'Step 3 - optional Define stages'. The main panel is titled 'API details' and contains the following information:

- API name:** wasif
- IP address type:** IPv4 (selected), Dualstack
- Integrations (1):** A single integration is configured for the Lambda function 'arn:aws:lambda:us-east-1:339163283124:function>HelloWorld' in the 'us-east-1' region, version '2.0'.

Resource Path:

/hello

Step 4: Create Method

Create Method:

GET

Integration Type:

Lambda Function

The screenshot shows the 'Configure routes - optional' step in the AWS API Gateway console. The left sidebar indicates the current step is 'Step 3 - optional Configure routes'. The main panel is titled 'Configure routes' and contains the following information:

- Method:** GET
- Resource path:** /hello
- Integration target:** HelloWorldFunction

Select:

HelloWorldFunction

Step 5: Deploy API

Actions → Deploy API

Stage Name:

dev

Deploy

Step 6: Test API

Copy Invoke URL

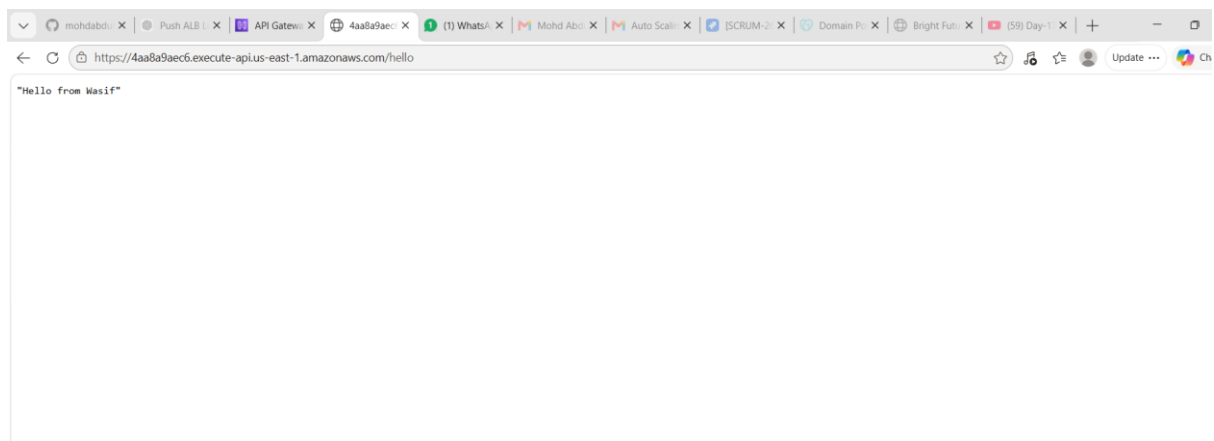
Example:

<https://abc123.execute-api.us-east-1.amazonaws.com/dev/hello>

Open in browser.

Expected:

Hello from Wasif



3. AWS KMS

Concept

[AWS KMS Documentation](#)

AWS KMS manages encryption keys securely.

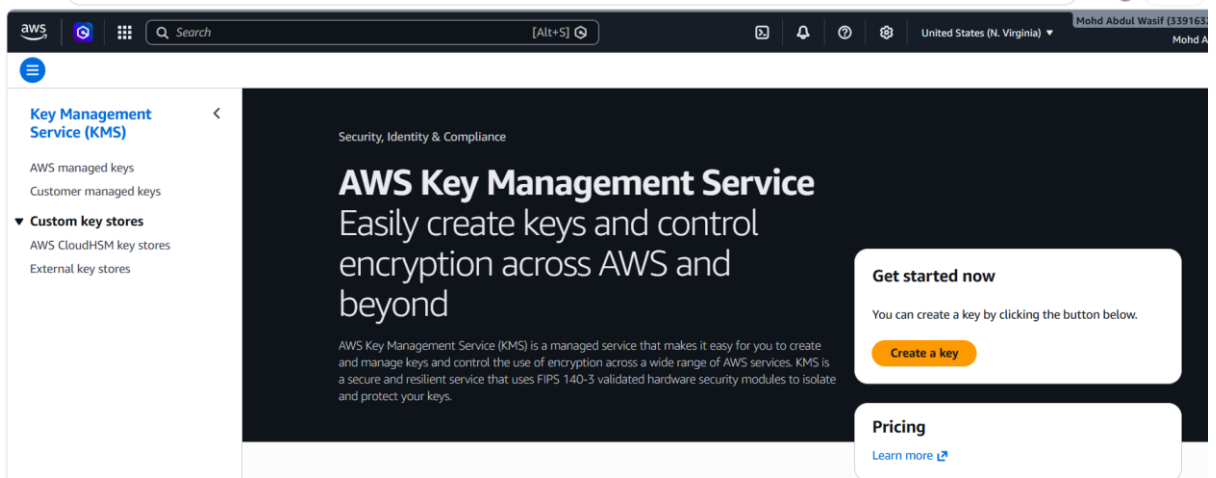
Use Cases

- Encrypt S3 Objects
- Encrypt EBS Volumes
- Encrypt RDS Databases
- Encrypt Secrets

Hands-On Implementation

Step 1: Create KMS Key

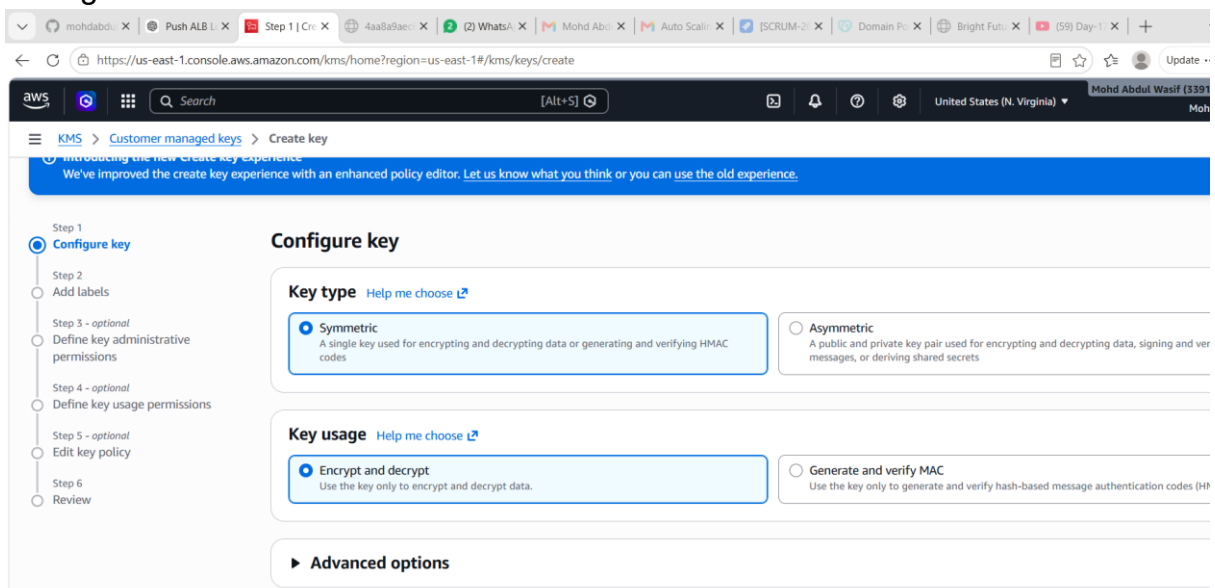
1. KMS Console



2. Customer Managed Keys

3. Create Key

Configuration:



Symmetric Encrypt and Decrypt

Step 2: Key Alias

Example:

my-kms-key

The screenshot shows the AWS Management Console interface for creating a new KMS key. The breadcrumb navigation at the top reads: KMS > Customer managed keys > Create key. On the left, a vertical list of steps is shown: Step 1: Configure key, Step 2: Add labels (highlighted with a blue circle), Step 3 - optional: Define key administrative permissions, Step 4 - optional: Define key usage permissions, Step 5 - optional: Edit key policy, Step 6: Review. The main content area is titled 'Add labels' and contains three sections: 'Alias' with a description 'You can change the alias at any time. Learn more' and a text input field containing 'my-kms-key'; 'Description - optional' with a description 'You can change the description at any time.' and a text input field containing 'my-kms-key'; and 'Tags - optional' which is currently empty.

Step 3: Key Administrators

Select your IAM user

Next → Finish

Step 4: Encrypt Text

KMS → Keys → my-kms-key

Cryptographic Operations

Plain Text:

Hello AWS KMS

Encrypt

Copy Cipher Text

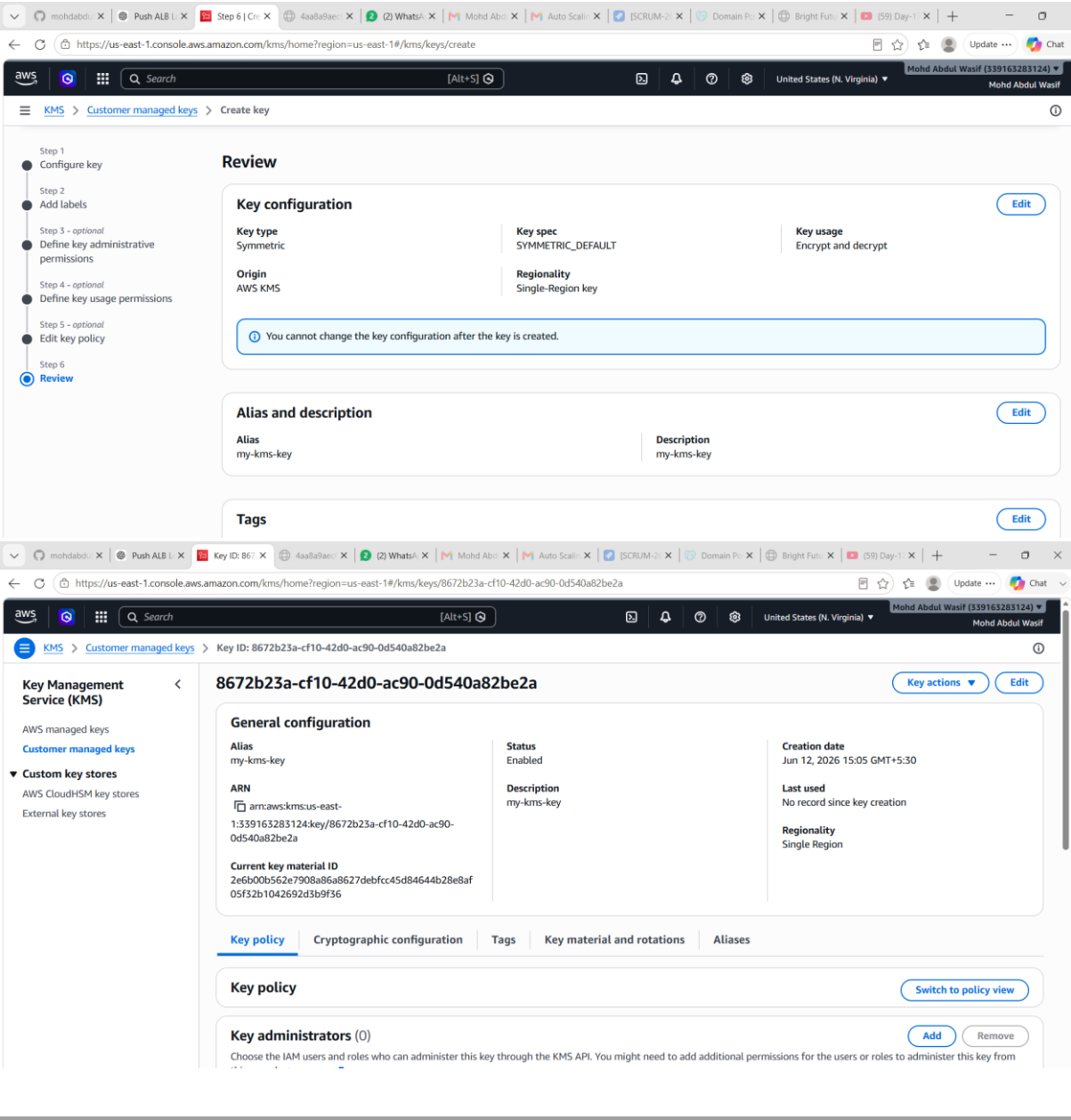
Step 5: Decrypt Text

Paste Cipher Text

Click Decrypt

Expected:

Hello AWS KMS



Architecture Diagram

User



API Gateway



Lambda Function

|

v

Response

KMS

|

v

Encryption/Decryption

Observations

Lambda

- Serverless compute service.
- Automatically scales.
- No server management required.

API Gateway

- Creates REST APIs.
- Integrates easily with Lambda.
- Secure and scalable.

KMS

- Centralized key management.
 - Supports encryption and decryption.
 - Integrates with multiple AWS services.
-

Final Result

Successfully implemented:

- ✓ AWS Lambda Function
- ✓ AWS API Gateway Integration
- ✓ AWS KMS Key Creation and Encryption
- ✓ Tested end-to-end API flow
- ✓ Verified encryption and decryption operations

Key Learnings

1. Learned how serverless computing works using AWS Lambda.
2. Understood API creation and management using API Gateway.
3. Learned how API Gateway invokes Lambda functions.
4. Understood encryption and key management using AWS KMS.
5. Learned how AWS services integrate to build secure and scalable applications.
6. Gained hands-on experience with serverless architecture.
7. Understood the advantages of managed AWS services over self-managed infrastructure.

Conclusion

Successfully implemented AWS Lambda, API Gateway, and AWS KMS. The Lambda function was deployed and tested successfully, API Gateway exposed the Lambda function through a REST API, and KMS was used to encrypt and decrypt data securely. This exercise provided practical experience in building a secure serverless application architecture on AWS.